IoT Practicals — Pins, Python Code & Terminal Commands

Organized from the supplied practical manuals and user-provided code. No theory/aim sections are included.

Accuracy note: Indentation has been normalized where possible. Some source code in PDFs is embedded as images and OCR may misread symbols. I have not claimed hardware-tested correctness. Check code and pinouts against your actual modules before powering them.

Practical 17 exception: includes pin connections and the supplied Node-RED JSON only, as requested.

Unavailable source: Complete source material was not supplied for every syllabus practical. Missing practicals are listed without invented connections or code.

Practical 1 — LED blinking patterns

Module pin / item	Raspberry Pi connection
LED 1 signal	BCM GPIO17; series resistor recommended
LED 2 signal	BCM GPIO18; series resistor recommended
LED 3 signal	BCM GPIO23; series resistor recommended
LED 4 signal	BCM GPIO22; series resistor recommended
LED cathodes	GND (each LED through its own resistor)

Python code (as supplied; indentation normalized):

```
import RPi.GPIO as GPIO
import time

led_pins = [17, 18, 23, 22]
GPIO.setmode(GPIO.BCM)

for pin in led_pins:
    GPIO.setup(pin, GPIO.OUT)

def all_on_off():
    for _ in range(5):
        for pin in led_pins:
            GPIO.output(pin, GPIO.HIGH)
        time.sleep(0.5)
        for pin in led_pins:
            GPIO.output(pin, GPIO.LOW)
        time.sleep(0.5)

def chase():
    for _ in range(5):
        for pin in led_pins:
            GPIO.output(pin, GPIO.HIGH)
            time.sleep(0.2)
            GPIO.output(pin, GPIO.LOW)

def blink_alternate():
    for _ in range(5):
        for i, pin in enumerate(led_pins):
            GPIO.output(pin, GPIO.HIGH if i % 2 == 0 else GPIO.LOW)
        time.sleep(0.5)
        for i, pin in enumerate(led_pins):
            GPIO.output(pin, GPIO.LOW if i % 2 == 0 else GPIO.HIGH)
        time.sleep(0.5)

try:
    while True:
        all_on_off()
        chase()
        blink_alternate()
except KeyboardInterrupt:
    GPIO.cleanup()
```

Terminal commands:

```
python3 practical1.py
```

Practical 2 — Telegram control of GPIO

Module pin / item	Raspberry Pi connection
Green LED positive	BCM GPIO2 / physical pin 3
Yellow LED positive	BCM GPIO3 / physical pin 5
Red LED positive	BCM GPIO4 / physical pin 7
Blue LED positive	BCM GPIO17 / physical pin 11
All LED negatives	GND (source manual says physical pin 9)

Python code (reconstructed from source; replace token):

```
import time
import datetime
import RPi.GPIO as GPIO
import telepot
from telepot.loop import MessageLoop

green = 2
yellow = 3
red = 4
blue = 17

GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)

for pin in [blue, yellow, red, green]:
    GPIO.setup(pin, GPIO.OUT)
    GPIO.output(pin, GPIO.LOW)

def action(msg):
    chat_id = msg["chat"]["id"]
    command = msg["text"].lower()
    print("Received: {}".format(command))

    if "on" in command:
        message = "On "
        if "blue" in command or "all" in command:
            GPIO.output(blue, GPIO.HIGH)
            message += "blue "
        if "yellow" in command or "all" in command:
            GPIO.output(yellow, GPIO.HIGH)
            message += "yellow "
        if "red" in command or "all" in command:
            GPIO.output(red, GPIO.HIGH)
            message += "red "
        if "green" in command or "all" in command:
            GPIO.output(green, GPIO.HIGH)
            message += "green "
        telegram_bot.sendMessage(chat_id, message + "light(s)")

    elif "off" in command:
        message = "Off "
        if "blue" in command or "all" in command:
            GPIO.output(blue, GPIO.LOW)
            message += "blue "
        if "yellow" in command or "all" in command:
            GPIO.output(yellow, GPIO.LOW)
            message += "yellow "
        if "red" in command or "all" in command:
            GPIO.output(red, GPIO.LOW)
            message += "red "
        if "green" in command or "all" in command:
            GPIO.output(green, GPIO.LOW)
            message += "green "
        telegram_bot.sendMessage(chat_id, message + "light(s)")

# Insert the Telegram bot token from BotFather.
telegram_bot = telepot.Bot("YOUR_BOT_TOKEN")
MessageLoop(telegram_bot, action).run_as_thread()

try:
    while True:
        time.sleep(10)
except KeyboardInterrupt:
    GPIO.cleanup()
```

Terminal commands:

```
sudo apt-get install idle
python3 -m pip install telepot
python3 test.py
```

Practical 3 — Display current time on TM1637 4-digit display

Module pin / item	Raspberry Pi connection
TM1637 GND	Physical pin 14 — GND (as source manual)
TM1637 VCC	Physical pin 4 — 5V
TM1637 DIO	Physical pin 18 — BCM GPIO24
TM1637 CLK	Physical pin 16 — BCM GPIO23

Python code (indentation and obvious OCR symbol errors normalized):

```
import time
import datetime
import RPi.GPIO as GPIO

CLK, DIO = 23, 24
HEX = [0x3F, 0x06, 0x5B, 0x4F, 0x66,
       0x6D, 0x7D, 0x07, 0x7F, 0x6F]

GPIO.setmode(GPIO.BCM)
GPIO.setup([CLK, DIO], GPIO.OUT)

def send_byte(b):
    for i in range(8):
        GPIO.output(CLK, 0)
        GPIO.output(DIO, (b >> i) & 1)
        GPIO.output(CLK, 1)
    GPIO.output(CLK, 0)
    GPIO.setup(DIO, GPIO.IN)
    GPIO.output(CLK, 1)
    GPIO.output(CLK, 0)
    GPIO.setup(DIO, GPIO.OUT)

def start_stop(func):
    GPIO.output(CLK, 1)
    GPIO.output(DIO, 1)
    GPIO.output(DIO, 0)
    GPIO.output(CLK, 0)
    func()
    GPIO.output(CLK, 0)
    GPIO.output(DIO, 0)
    GPIO.output(CLK, 1)
    GPIO.output(DIO, 1)

try:
    while True:
        now = datetime.datetime.now()
        digits = [now.hour // 10, now.hour % 10,
                  now.minute // 10, now.minute % 10]
        start_stop(lambda: send_byte(0x40))

        def write_digits():
            send_byte(0xC0)
            for i, digit in enumerate(digits):
                seg = HEX[digit] | (0x80 if i == 1 and now.second % 2 == 0 else 0)
                send_byte(seg)

        start_stop(write_digits)
        start_stop(lambda: send_byte(0x8A))
        time.sleep(0.5)
except KeyboardInterrupt:
    GPIO.cleanup()
```

Terminal commands:

```
python3 test.py
```

Practical 4 — GPS module / UART receive and echo

Module pin / item	Raspberry Pi connection
GPS TX	Raspberry Pi UART RXD — physical pin 10 / BCM GPIO15
GPS RX	Raspberry Pi UART TXD — physical pin 8 / BCM GPIO14, if needed
GPS power/GND	Use the GPS module's specified supply and common GND; exact power pins are not present in the s

Python code supplied in chat:

```
#!/usr/bin/python
import time
import RPi.GPIO as GPIO
import pio
import Ports

GPIO.setmode(GPIO.BOARD)
GPIO.setwarnings(False)
pio.uart = Ports.UART()  # Define serial port

while True:
    data = pio.uart.recv()
    pio.uart.print(data)
```

Note: this is the exact UART receive/echo pattern supplied in chat. It depends on pio and Ports modules; the source does not establish that this parses GPS NMEA data.

Practical 5 — ADS1115 oscilloscope

Module pin / item	Raspberry Pi connection
ADS1115 VDD	Physical pin 1 — 3.3V (as source table)
ADS1115 GND	Physical pin 9 — GND (as source table)
ADS1115 SCL	Physical pin 5 — BCM GPIO3 / SCL
ADS1115 SDA	Physical pin 3 — BCM GPIO2 / SDA
Analog signal	ADS1115 input channel P0; stay within ADC input-voltage limits

Python code:

```
import time
import board
import busio
import adafruit_ads1x15.ads1015 as ADS
from adafruit_ads1x15.analog_in import AnalogIn
import matplotlib.pyplot as plt

# Setup I2C and ADC
i2c = busio.I2C(board.SCL, board.SDA)
ads = ADS.ADS1015(i2c)
chan = AnalogIn(ads, ADS.P0)

# Prepare lists to store data
times = []
voltages = []

start_time = time.time()

plt.ion()  # Turn interactive mode on for live plot
fig, ax = plt.subplots()

while True:
    current_time = time.time() - start_time
    voltage = chan.voltage

    # Append data
    times.append(current_time)
    voltages.append(voltage)

    # Plot data
    ax.clear()
    ax.plot(times, voltages, label="Voltage (V)")
    ax.set_xlabel("Time (s)")
    ax.set_ylabel("Voltage (V)")
    ax.set_title("ADS1015 Voltage Readings")
    ax.legend()
    plt.pause(0.1)

    time.sleep(0.5)
```

Terminal commands:

```
sudo raspi-config  # enable I2C
sudo apt-get update
sudo apt-get upgrade
sudo apt-get install build-essential python3-dev python3-smbus git
cd ~
git clone https://github.com/adafruit/Adafruit_Python_ADS1x15.git
cd Adafruit_Python_ADS1x15
sudo python3 setup.py install
sudo apt-get install python-matplotlib python-pip
sudo pip install drawnow
cd examples
python3 sampletest.py
```

Practical 6 — 16×2 LCD using I2C

Module pin / item	Raspberry Pi connection
LCD GND	Physical pin 6 — GND
LCD VCC	Physical pin 4 — 5V
LCD SDA	Physical pin 3 — BCM GPIO2 / SDA
LCD SCL	Physical pin 5 — BCM GPIO3 / SCL

Python code:

```
from RPLCD.i2c import CharLCD
from time import sleep

lcd = CharLCD("PCF8574", 0x27)  # Use 0x3F if detected
lcd.write_string("Hello from Pi 400!")
sleep(3)
lcd.clear()
```

Terminal commands:

```
sudo raspi-config  # enable I2C
sudo apt update
sudo apt install i2c-tools python3-smbus python3-pip
pip3 install RPLCD
i2cdetect -y 1
python3 lcd_test.py
```

Practical 7 — RFID read/write

Module pin / item	Raspberry Pi connection
RC522 3.3V	Physical pin 1 — 3.3V
RC522 GND	Physical pin 6 — GND
RC522 RST	Physical pin 22 — BCM GPIO25
RC522 MISO	Physical pin 21 — BCM GPIO9
RC522 MOSI	Physical pin 19 — BCM GPIO10
RC522 SCK	Physical pin 23 — BCM GPIO11
RC522 SDA / SS	Physical pin 24 — BCM GPIO8

Python code:

```
from mfrc522 import SimpleMFRC522
import RPi.GPIO as GPIO

reader = SimpleMFRC522()


def main():
  try:
    print('=== RFID Manager ===')
    print('1. Write Data to Tag')
    print('2. Read Data from Tag')
    print('3. Erase/Wipe Tag Data')
    choice = input('Enter your choice (1, 2, or 3): ').strip()
    print('-' * 30)

    if choice == '1':
      text = input('Enter new data to write: ')
      print('Place tag near the reader...')
      reader.write(text)
      print('SUCCESS: Data written successfully!')
    elif choice == '2':
      print('Place the RFID tag near the reader...')
      id, text = reader.read()
      print(f'Tag ID: {id}')
      print(f'Data stored: {text.strip()}')
    elif choice == '3':
      print('Hold your RFID tag near the reader to erase data...')
      reader.write('')
      print('SUCCESS: Tag data wiped successfully!')
    else:
      print(
          'INVALID CHOICE: Please run the script again and select 1, 2, or 3.'
      )
  except Exception as e:
    print(f'An error occurred: {e}')
  finally:
    GPIO.cleanup()


if __name__ == '__main__':
  main()
```

Practical 7 — RFID read/write (continued)

Terminal commands:

```
sudo apt update
sudo apt install -y python3-pip python3-spidev
python3 -m pip install mfrc522
sudo raspi-config  # enable SPI
sudo reboot
nano rfid_manager.py
python3 rfid_manager.py
```

Practical 8 — Fingerprint sensor interfacing

Module pin / item	Raspberry Pi connection
Fingerprint VCC (red)	Physical pin 4 — 5V (source manual)
Fingerprint GND (black)	Physical pin 6 or 9/14 — GND; follow the specific manual/module
Sensor TXD	Physical pin 10 — BCM GPIO15 / UART RXD
Sensor RXD	Physical pin 8 — BCM GPIO14 / UART TXD

Fingerprint enrollment code:

```
import time
import serial
import adafruit_fingerprint

uart = serial.Serial("/dev/serial0", baudrate=57600, timeout=1)
finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)
finger_id = int(input("Enter fingerprint ID (1-127): "))

print("Place finger on sensor...")
while finger.get_image() != adafruit_fingerprint.OK:
    time.sleep(0.1)

if finger.image_2_tz(1) != adafruit_fingerprint.OK:
    print("Image conversion failed")
    raise SystemExit

print("Remove finger")
time.sleep(2)
print("Place the same finger again...")
while finger.get_image() != adafruit_fingerprint.OK:
    time.sleep(0.1)

if finger.image_2_tz(2) != adafruit_fingerprint.OK:
    print("Image conversion failed")
    raise SystemExit
if finger.create_model() != adafruit_fingerprint.OK:
    print("Fingerprints did not match")
    raise SystemExit

if finger.store_model(finger_id) == adafruit_fingerprint.OK:
    print("Fingerprint enrolled successfully")
    print("Stored ID:", finger_id)
else:
    print("Failed to store fingerprint")
```

Terminal commands:

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-serial
pip3 install adafruit-circuitpython-fingerprint
sudo raspi-config  # Serial Port: login shell No, hardware serial Yes
sudo reboot
ls -l /dev/serial0
python3 enroll.py
```

Practical 9 — Pi Camera capture image and video

No Pi Camera practical source PDF or code was supplied. No code or wiring has been invented.

Practical 10 — IoT home automation

No dedicated home-automation practical source was supplied. No connections or code have been invented.

Practical 11 — Fingerprint sensor + 16×2 I2C LCD

Module pin / item	Raspberry Pi connection
LCD GND / VCC	Physical pin 6 GND / physical pin 2 or 4 5V per source table
LCD SDA / SCL	Physical pin 3 GPIO2 SDA / physical pin 5 GPIO3 SCL
Fingerprint VCC / GND	5V / GND
Fingerprint TXD	Physical pin 10 — GPIO15 RXD
Fingerprint RXD	Physical pin 8 — GPIO14 TXD

Enrollment code:

```
import time
import serial
import adafruit_fingerprint

uart = serial.Serial("/dev/serial0", baudrate=57600, timeout=1)
finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)
location = int(input("Enter Slot ID (1-127): "))

print("Place finger on sensor...")
while finger.get_image() != adafruit_fingerprint.OK:
    pass
finger.image_2_tz(1)

print("Remove finger...")
time.sleep(2)
while finger.get_image() == adafruit_fingerprint.OK:
    pass

print("Place SAME finger again...")
while finger.get_image() != adafruit_fingerprint.OK:
    pass
finger.image_2_tz(2)

if finger.create_model() == adafruit_fingerprint.OK:
    if finger.store_model(location) == adafruit_fingerprint.OK:
        print(f"Enrolled successfully at Slot {location}!")
    else:
        print("Error saving template.")
else:
    print("Fingerprint mismatch. Try again.")
```

Verification code (from the source manual; indentation normalized):

```
import time
import serial
import adafruit_fingerprint
from RPLCD.i2c import CharLCD

lcd = CharLCD("PCF8574", 0x27, cols=16, rows=2)
uart = serial.Serial("/dev/serial0", baudrate=57600, timeout=1)
finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)

def show(line1, line2=""):
    lcd.clear()
    lcd.cursor_pos = (0, 0)
    lcd.write_string(line1[:16])
    lcd.cursor_pos = (1, 0)
    lcd.write_string(line2[:16])

show("System Ready", "Place Finger")
while True:
    if finger.get_image() == adafruit_fingerprint.OK:
        show("Processing...", "Please Wait")
        if finger.image_2_tz(1) == adafruit_fingerprint.OK:
            if finger.finger_search() == adafruit_fingerprint.OK:
                show("RFID UID MATCHED", f"ID: {finger.finger_id}")
                time.sleep(3)
            else:
                show("RFID UID", "UNMATCHED")
                time.sleep(1.5)
                show("Access Denied", "")
                time.sleep(2)
        show("Place Finger", "")
    time.sleep(0.1)
```

Terminal commands:

```
sudo apt-get update && sudo apt-get install -y i2c-tools python3-pip python3-serial
pip3 install RPLCD adafruit-circuitpython-fingerprint smbus2
i2cdetect -y 1
python3 enroll.py
python3 main_verify.py
```

Practical 12 — Fingerprint authenticated door lock

Module pin / item	Raspberry Pi connection
Relay VCC / GND	5V physical pin 2 / GND physical pin 9 (source table)
Relay IN	BCM GPIO17 / physical pin 11
Fingerprint VCC / GND	5V physical pin 2 / GND physical pin 6
Fingerprint TX	GPIO15 RXD / physical pin 10
Fingerprint RX	GPIO14 TXD / physical pin 8

Enrollment code:

```
import serial
import time
import adafruit_fingerprint

uart = serial.Serial("/dev/serial0", baudrate=57600, timeout=1)
finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)
finger_id = int(input("Enter fingerprint ID (1-127): "))

print("Place finger on sensor...")
while finger.get_image() != adafruit_fingerprint.OK:
    time.sleep(0.1)
if finger.image_2_tz(1) != adafruit_fingerprint.OK:
    print("Image conversion failed")
    raise SystemExit

print("Remove finger")
time.sleep(2)
print("Place the same finger again...")
while finger.get_image() != adafruit_fingerprint.OK:
    time.sleep(0.1)
if finger.image_2_tz(2) != adafruit_fingerprint.OK:
    print("Image conversion failed")
    raise SystemExit
if finger.create_model() != adafruit_fingerprint.OK:
    print("Fingerprints did not match")
    raise SystemExit
if finger.store_model(finger_id) == adafruit_fingerprint.OK:
    print("Fingerprint enrolled successfully")
else:
    print("Failed to store fingerprint")
```

Door-lock verification code (core from source; relay active-LOW):

```
import serial
import time
import RPi.GPIO as GPIO
import adafruit_fingerprint

GPIO.setwarnings(False)
RELAY_PIN = 17
RELAY_ON = GPIO.LOW
RELAY_OFF = GPIO.HIGH
GPIO.setmode(GPIO.BCM)
GPIO.setup(RELAY_PIN, GPIO.OUT, initial=RELAY_OFF)

try:
    uart = serial.Serial("/dev/serial0", baudrate=57600, timeout=1)
    finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)
    print("Fingerprint Door Lock System")
    print("Place Finger")
    while True:
        if finger.get_image() == adafruit_fingerprint.OK:
            print("Processing...")
            if finger.image_2_tz(1) != adafruit_fingerprint.OK:
                print("Fingerprint processing failed")
                time.sleep(1)
                continue
            if finger.finger_search() == adafruit_fingerprint.OK:
                print("Access Granted")
                GPIO.output(RELAY_PIN, RELAY_ON)
                print("Door Unlocked")
                time.sleep(5)
                GPIO.output(RELAY_PIN, RELAY_OFF)
                print("Door Locked")
            else:
                print("Access Denied")
            time.sleep(1)
except Exception as e:
    print("Communication error:", e)
finally:
    GPIO.output(RELAY_PIN, RELAY_OFF)
    GPIO.cleanup()
```

Terminal commands:

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-serial
pip3 install adafruit-circuitpython-fingerprint
sudo raspi-config  # enable UART; disable login shell, enable hardware serial
sudo reboot
ls -l /dev/serial0
python3 enroll.py
python3 door_lock.py
```

Practical 13 — MQTT publisher-subscriber LED control

Module pin / item	Raspberry Pi connection
LED positive	Physical pin 12 — BCM GPIO18 (source pin table; code uses BOARD pin 12)
LED negative	GND physical pin 6

Python code (indentation normalized):

```
import paho.mqtt.client as mqtt
import RPi.GPIO as GPIO

GPIO.setmode(GPIO.BOARD)
GPIO.setwarnings(False)
LED = 12
GPIO.setup(LED, GPIO.OUT, initial=GPIO.LOW)

def on_connect(client, userdata, flags, reason_code, properties):
    print("Connected with result code:", reason_code)
    client.subscribe("led")
    print("Subscribed to topic: led")

def on_message(client, userdata, msg):
    message = msg.payload.decode("utf-8").strip().lower()
    print("Received:", message)
    if message == "on":
        GPIO.output(LED, GPIO.HIGH)
        print("LED ON")
    elif message == "off":
        GPIO.output(LED, GPIO.LOW)
        print("LED OFF")
    else:
        print("Unknown message:", message)

client = mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message
client.connect("broker.emqx.io", 1883, 60)
print("Waiting for MQTT messages...")
try:
    client.loop_forever()
finally:
    GPIO.cleanup()
```

Terminal commands:

```
python3 -m venv myenv
source myenv/bin/activate
pip install paho-mqtt
python3 test.py
```

Practical 14 — Flask web dashboard to control relay

Module pin / item	Raspberry Pi connection
Relay VCC	5V physical pin 2 (source table)
Relay GND	GND physical pin 6
Relay IN / CH1	BCM GPIO18 / physical pin 12 in source pin table; source code uses BCM RELAY=17, which conflic

Python code (indentation normalized; source's GPIO/pin mismatch flagged above):

```
from flask import Flask, render_template_string
import RPi.GPIO as GPIO

app = Flask(__name__)
RELAY = 17

GPIO.setmode(GPIO.BCM)
GPIO.setup(RELAY, GPIO.OUT)
GPIO.output(RELAY, GPIO.HIGH)

HTML = """
<!DOCTYPE html>
<html>
<head>
    <title>Relay Control</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
</head>
<body>
    <h1>Raspberry Pi Relay</h1>
    {% if state %}
        <div>Status: ON</div>
    {% else %}
        <div>Status: OFF</div>
    {% endif %}
    <a href="/on"><button>ON</button></a>
    <a href="/off"><button>OFF</button></a>
</body>
</html>
"""

@app.route("/")
def home():
    state = GPIO.input(RELAY) == GPIO.LOW
    return render_template_string(HTML, state=state)

@app.route("/on")
def turn_on():
    GPIO.output(RELAY, GPIO.LOW)
    return render_template_string(HTML, state=True)

@app.route("/off")
def turn_off():
    GPIO.output(RELAY, GPIO.HIGH)
    return render_template_string(HTML, state=False)

try:
    app.run(host="0.0.0.0", port=5000)
finally:
    GPIO.cleanup()
```

Terminal commands:

```
sudo apt update
sudo apt install python3-flask
python3 -c "import flask; print('Flask OK')"
nano relay.py
python3 relay.py
# In another terminal:
hostname -I
```

Practical 15 — Two-factor authentication using fingerprint + RFID + relay

No separate complete source for Practical 15 was supplied. Practical 19 has related RFID/fingerprint access-control source, but it is not silently substituted as Practical 15.

Practical 16 — Motion detector using camera + 16×2 LCD

No source PDF or complete code for Practical 16 was supplied.

Practical 17 — Node-RED physical button + LED

Module pin / item	Raspberry Pi connection
Button side 1	Physical pin 1 — 3.3V
Button side 2 / input	Physical pin 11 — BCM GPIO17; input pull-down enabled
LED positive through 220–330 Ω resistor	Physical pin 13 — BCM GPIO27
LED negative	Physical pin 6 — GND

Node-RED flow JSON supplied in chat (kept as JSON; no Python code or terminal commands in this practical section):


```
[
  {
    "id": "34d800aeebc94007",
    "type": "tab",
    "label": "Button LED - Diagram Flow",
    "disabled": false,
    "info": "GPIO17 button input, GPIO27 LED output. Button is wired from GPIO17 to 3V3 and input pull-down is enabled."
  },
  {
    "id": "f5954e81414a49e8",
    "type": "inject",
    "z": "34d800aeebc94007",
    "name": "Initialize state (0)",
    "props": [
      {
        "p": "payload"
      }
    ],
    "repeat": "",
    "crontab": "",
    "once": true,
    "onceDelay": 0.5,
    "topic": "",
    "payload": "0",
    "payloadType": "num",
    "x": 1170,
    "y": 300,
    "wires": [
      [
        "7a7651d05df141a5"
      ]
    ]
  },
  {
    "id": "b4d0eeba3501480b",
    "type": "rpi-gpio in",
    "z": "34d800aeebc94007",
    "name": "GPIO IN 17",
    "pin": "17",
    "intype": "down",
    "debounce": "25",
    "read": false,
    "bcm": true,
    "x": 150,
    "y": 150,
    "wires": [
      [
        "bea2eef241a44208"
      ]
    ]
  },
  {
    "id": "bea2eef241a44208",
    "type": "switch",
    "z": "34d800aeebc94007",
    "name": "SWITCH (1)",
    "property": "state",
    "propertyType": "global",
    "rules": [
      {
        "t": "eq",
        "v": "0",
        "vt": "num"
      },
      {
        "t": "eq",
        "v": "1",
        "vt": "num"
      }
    ],
    "checkall": "true",
    "repair": false,
    "outputs": 2,
    "x": 490,
    "y": 160,
```

```
  "wires": [
    [
      "1abf20d5cbc6490b"
    ],
    [
      "1abf20d5cbc6490b"
    ]
  ]
},
{
  "id": "1abf20d5cbc6490b",
  "type": "switch",
  "z": "34d800aeebc94007",
  "name": "SWITCH (2)",
  "property": "payload",
  "propertyType": "msg",
  "rules": [
    {
      "t": "eq",
      "v": "1",
      "vt": "num"
    }
  ],
  "checkall": "true",
  "repair": false,
  "outputs": 1,
  "x": 850,
  "y": 140,
  "wires": [
    [
      "cbe298e9cd324af3"
    ]
  ]
},
{
  "id": "cbe298e9cd324af3",
  "type": "switch",
  "z": "34d800aeebc94007",
  "name": "SWITCH (3)",
  "property": "state",
  "propertyType": "global",
  "rules": [
    {
      "t": "eq",
      "v": "0",
      "vt": "num"
    },
    {
      "t": "eq",
      "v": "1",
      "vt": "num"
    }
  ],
  "checkall": "true",
  "repair": false,
  "outputs": 2,
  "x": 1150,
  "y": 140,
  "wires": [
    [
      "5b54793579864525"
    ],
    [
      "7a7651d05df141a5"
    ]
  ]
},
{
  "id": "5b54793579864525",
  "type": "change",
  "z": "34d800aeebc94007",
  "name": "CHANGE (1)",
  "rules": [
    {
      "t": "set",
      "p": "state",
      "pt": "global",
      "to": "1",
      "tot": "num"
    },
    {
      "t": "set",
      "p": "payload",
      "pt": "msg",
      "to": "1",
      "tot": "num"
    }
  ],
  "action": "",
  "property": "",
  "from": "",
  "to": "",
  "reg": false,
  "x": 1430,
  "y": 40,
```

```
    "wires": [
      [
        "223feab01d594e51"
      ]
    ]
  },
  {
    "id": "7a7651d05df141a5",
    "type": "change",
    "z": "34d800aeebc94007",
    "name": "CHANGE (2)",
    "rules": [
      {
        "t": "set",
        "p": "state",
        "pt": "global",
        "to": "0",
        "tot": "num"
      },
      {
        "t": "set",
        "p": "payload",
        "pt": "msg",
        "to": "0",
        "tot": "num"
      }
    ],
    "action": "",
    "property": "",
    "from": "",
    "to": "",
    "reg": false,
    "x": 1430,
    "y": 220,
    "wires": [
      [
        "223feab01d594e51"
      ]
    ]
  },
  {
    "id": "223feab01d594e51",
    "type": "rpi-gpio out",
    "z": "34d800aeebc94007",
    "name": "GPIO OUT 27",
    "pin": "27",
    "set": "",
    "level": "0",
    "freq": "",
    "out": "out",
    "bcm": true,
    "x": 1750,
    "y": 140,
    "wires": []
  },
  {
    "id": "83008c98cee4d73c",
    "type": "global-config",
    "env": [],
    "modules": {
      "node-red-node-pi-gpio": "2.0.7"
    }
  }
]
```

Practical 18 — CPU%, memory% and CPU temperature

No external GPIO pin connections are required for this software-only practical.

Python code (indentation and obvious OCR errors normalized):

```
#!/usr/bin/env python3
import psutil
import time

def get_cpu_temperature():
    """Read Raspberry Pi CPU temperature."""
    try:
        with open("/sys/class/thermal/thermal_zone0/temp", "r") as file:
            temp = int(file.read()) / 1000.0
        return temp
    except (FileNotFoundError, ValueError):
        return None

while True:
    cpu_usage = psutil.cpu_percent(interval=1)
    memory_usage = psutil.virtual_memory().percent
    cpu_temp = get_cpu_temperature()

    print("--------------------------------")
    print(f"CPU Usage       : {cpu_usage:.1f}%")
    print(f"Memory Usage    : {memory_usage:.1f}%")
    if cpu_temp is not None:
        print(f"CPU Temperature : {cpu_temp:.1f} °C")
    else:
        print("CPU Temperature : Not available")
    print("--------------------------------")
    time.sleep(1)
```

Terminal commands:

```
sudo apt update
sudo apt install python3-psutil
python3 test.py
```

Practical 19 — Embedded RFID/fingerprint access control

Module pin / item	Raspberry Pi connection
Relay VCC / GND / IN	5V physical pin 2 / GND physical pin 9 / BCM GPIO23 physical pin 16
LCD GND / VCC	Physical pin 6 GND / physical pin 4 5V
LCD SDA / SCL	Physical pin 3 GPIO2 SDA / physical pin 5 GPIO3 SCL
Fingerprint VCC / GND	5V / GND
Fingerprint TX / RX	GPIO15 RXD physical pin 10 / GPIO14 TXD physical pin 8
RC522 3.3V / GND	Physical pin 1 / physical pin 6
RC522 RST	Physical pin 22 — BCM GPIO25
RC522 MISO / MOSI / SCK / SDA-SS	Physical pins 21 / 19 / 23 / 24 — BCM GPIO9 / GPIO10 / GPIO11 / GPIO8
LED	BCM GPIO21 per source code; physical pin not specified in source table

Access-control Python code (source code reorganized for indentation; verify against original before running):

```
import RPi.GPIO as GPIO
import time
import csv
import os
import serial
from datetime import datetime
from RPLCD.i2c import CharLCD
from mfrc522 import SimpleMFRC522
import adafruit_fingerprint

RELAY = 23
LED = 21
GPIO.setmode(GPIO.BCM)
GPIO.setup(RELAY, GPIO.OUT)
GPIO.setup(LED, GPIO.OUT)
GPIO.output(RELAY, GPIO.HIGH)  # Active-LOW relay: HIGH = locked
GPIO.output(LED, GPIO.LOW)

lcd = CharLCD(i2c_expander="PCF8574", address=0x27,
              port=1, cols=16, rows=2, charmap="A00",
              auto_linebreaks=True)
reader = SimpleMFRC522()
WHITELIST = [123456789, 987654321]  # Replace with authorized UIDs

uart = serial.Serial("/dev/serial0", baudrate=57600, timeout=1)
finger = adafruit_fingerprint.Adafruit_Fingerprint(uart)
LOG_FILE = "access_log.csv"

def log_event(method, result):
    new_file = not os.path.exists(LOG_FILE)
    with open(LOG_FILE, "a", newline="") as file:
        writer = csv.writer(file)
        if new_file:
            writer.writerow(["Time", "Method", "Result"])
        writer.writerow([datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                         method, result])

def show_message(line1, line2=""):
    lcd.clear()
    lcd.write_string(line1[:16])
    if line2:
        lcd.cursor_pos = (1, 0)
        lcd.write_string(line2[:16])

def unlock(method):
    show_message("ACCESS GRANTED", method)
    GPIO.output(LED, GPIO.HIGH)
    GPIO.output(RELAY, GPIO.LOW)
    log_event(method, "GRANTED")
    time.sleep(5)
    GPIO.output(RELAY, GPIO.HIGH)
    GPIO.output(LED, GPIO.LOW)

def deny(method):
    show_message("ACCESS DENIED", method)
    GPIO.output(LED, GPIO.LOW)
    log_event(method, "DENIED")
    time.sleep(2)

def check_fingerprint():
    show_message("Place Finger")
    start = time.time()
    while time.time() - start < 5:
```

```
        result = finger.get_image()
        if result == adafruit_fingerprint.OK:
            break
        if result == adafruit_fingerprint.NOFINGER:
            time.sleep(0.1)
            continue
        return False
    else:
        return False
    show_message("Processing...")
    if finger.image_2_tz(1) != adafruit_fingerprint.OK:
        return False
    return finger.finger_search() == adafruit_fingerprint.OK

try:
    show_message("ACCESS CONTROL", "SYSTEM READY")
    time.sleep(2)
    while True:
        show_message("Scan RFID", "or Fingerprint")
        card_id, text = reader.read_no_block()
        if card_id:
            if card_id in WHITELIST:
                unlock("RFID")
            else:
                deny("RFID")
            time.sleep(1)
            continue
        if check_fingerprint():
            unlock("FINGERPRINT")
        time.sleep(1)
except KeyboardInterrupt:
    print("\nProgram stopped")
finally:
    GPIO.output(RELAY, GPIO.HIGH)
    GPIO.output(LED, GPIO.LOW)
    GPIO.cleanup()
    lcd.clear()
```

Terminal commands:

```
sudo apt update
sudo apt install -y python3-pip python3-serial python3-rpi.gpio i2c-tools
pip3 install RPLCD adafruit-circuitpython-fingerprint mfrc522
sudo raspi-config  # enable I2C and UART
sudo i2cdetect -y 1
python3 rfid_test.py
python3 enroll.py
python3 access_control.py
```

Practical 20 — Windows IoT Core demo

No Windows IoT Core practical manual, pinout, code or terminal commands were supplied in the uploaded sources.

Source gaps

Complete source files were not supplied for Practical 7 (RFID read/write), Practical 9 (Pi Camera), Practical 10 (home automation), Practical 15 (two-factor authentication as a separate practical), Practical 16 (camera + LCD motion detector), and Practical 20 (Windows IoT Core). These are intentionally not filled with guessed code or pinouts.

Practical 17 JSON caveat: the provided JSON has been preserved and formatted, but the flow was not hardware-tested and may not fully match the syllabus requirement.